

SHL Conversational Assessment Recommender

Overview

This project implements a stateless FastAPI service that recommends SHL assessments through conversation. The service exposes two endpoints:

- `GET /health` — returns `{"status": "ok"}`
- `POST /chat` — accepts full conversation history, returns agent reply + structured recommendations

The system is built around the evaluator's constraints: strict response schema, stateless requests, 8-turn cap, 30-second timeout per call, and catalog-grounded recommendations only. The key design choice was to keep the API behavior deterministic at the schema level while allowing the LLM to improve reply quality.

Stack And Tools Used

- **FastAPI** for the API layer (lightweight, schema-first, native async support)
- **Requests** + **BeautifulSoup** for scraping the SHL product catalog
- **LangChain** + **DeepSeek V4 Flash** for the optional LLM-powered agent layer (OpenAI-compatible, free tier), I used paid version **DeepSeek V4 Flash** model to test this agent
- **Render free tier** for deployment (explicitly listed as a recommended platform in the assignment)
- **AI-assisted development**: Claude was used for code scaffolding, test generation, and approach document drafting. All design decisions were reviewed and justified manually.

Data And Catalog Setup

A scraper (`scraper.py`) crawls the SHL assessments section, visits ~80 pages, and identifies 10 individual assessment detail pages by filtering out category hubs and framework pages. The scraper stores:

- Raw HTML pages in `data/raw/shl_pages/`
- A crawl manifest in `data/raw/shl_catalog_manifest.json`
- A normalized catalog in `data/processed/shl_catalog.json` (10 items)

Each catalog entry stores: name, SHL URL, test type (K/C/P/B), description, categories, skills, job levels, and aliases. This gives the API a local, queryable catalog — no live SHL calls needed during recommendations.

Retrieval And Ranking

Retrieval is implemented as weighted heuristic ranking over the normalized catalog (`catalog_service.py`), avoiding a vector database in favor of transparency and ease of

debugging. The scoring function considers:

- Skill overlap (3.0x weight)
- Name overlap (2.5x weight)
- Category overlap (2.0x weight)
- Job level overlap (1.5x weight)
- Full-text overlap (1.0x weight)
- Exact name match bonus (4.0x)
- Seniority alignment (+/- score)
- Domain intent matching (technical, personality, cognitive)

A diversification step ensures mixed-intent queries (e.g., "Java developer with stakeholder skills") return a balanced shortlist spanning both technical and personality assessments. Results are capped at 5 (configurable via `MAX_RECOMMENDATIONS`).

Conversation Design

The chat service (`chat_service.py`) is stateless: every `/chat` call carries the full conversation history as a `messages` array. The backend:

1. Extracts all user messages and tokenizes them, filtering out generic words
2. Builds a lightweight `ConversationContext` with informative tokens, seniority hints, and user intent
3. Runs guardrails (injection, legal, off-topic detection)
4. Routes to the appropriate handler: clarify, recommend, or compare
5. Falls back to the LangChain agent for richer reply generation when an LLM is configured

The `min_context_signals` threshold (default: 2) controls when the system clarifies vs. recommends. A single-word query like "I need an assessment" has minimal informative tokens and triggers a clarifying question.

Four Conversational Behaviors

1. **Clarify** — Vague queries below `min_context_signals` receive a focused question about role, seniority, and assessment type
2. **Recommend** — When enough context exists, returns 1-10 items with catalog names and URLs, plus an LLM-generated or template-based reply
3. **Refine** — Since all user messages are pooled to build context, new constraints (e.g., "add personality tests") update the token profile and produce a revised shortlist without restarting
4. **Compare** — Detects comparison keywords ("vs", "difference", "compare"), matches assessment names/aliases against the catalog, and generates a grounded comparison using catalog descriptions

Safety And Scope Control

A guardrail layer (`guardrails.py`) blocks three categories of off-scope queries:

- Prompt-injection attempts (keyword-based: "ignore previous instructions", "reveal system prompt", "jailbreak", "bypass your rules")
- Legal/compliance advice queries
- Unrelated topics (weather, restaurants, flights, movie recommendations)

Blocked queries return `end_of_conversation: true` to signal the evaluator that the agent has refused. This aligns with the assignment's behavior probes for scope enforcement.

LLM Integration

An optional LangChain agent (`langchain_agent.py`) wraps an OpenAI-compatible LLM (DeepSeek V4 Flash by default). The agent is given two tools: `search_shl_catalog` and `get_assessment_details`, both backed by the local catalog — ensuring all model output stays grounded. The system prompt explicitly instructs the agent to stay within SHL assessment selection, refuse off-topic queries, and ground answers in tool results.

The LLM is **optional**: if `SHL_AGENT_MODEL` is empty, the service falls back to template-based replies. The core recommendation and comparison paths work without it. This was intentional — the evaluator depends on schema correctness, so the LLM improves wording but doesn't control the response structure.

Evaluation Approach

Automated API tests (`tests/test_api.py`) cover five scenarios:

Test	Behavior Verified
<code>test_health</code>	Returns <code>{"status": "ok"}</code> with HTTP 200
<code>test_chat_clarifies_vague_request</code>	"I need an assessment" → empty recommendations + clarification
<code>test_chat_recommends_for_groundquery</code>	"personality assessment for stakeholder-facing hires" → 1-10 recommendations containing personality items
<code>test_chat_refuses_prompt_injection</code>	"Ignore previous instructions..." → blocked, <code>end_of_conversation: true</code>
<code>test_chat_compares_alias_names</code>	"What is the difference between OPQ and GSA?" → grounded comparison with catalog names in reply

All 5 tests pass. Further evaluation would benefit from replay tests against the 10 public conversation traces, measuring Recall@10 and behavioral probe pass rates.

What Didn't Work And How It Was Improved

1. **Sample-only catalog** — The initial scaffold used placeholder data with fake URLs. This made recommendations weak and unverifiable. Fixed by building a real scraper that crawls the SHL website and extracts 10 individual assessment pages with actual SHL URLs, descriptions, and metadata.
2. **Overly broad page classification** — Early scraper versions treated category hub pages (e.g., "Cognitive Assessments") as individual assessments, while missing real product detail pages. Fixed by adding path-depth checks and excluding pages that have sub-pages linked beneath them.
3. **Test-fixture mismatch** — The comparison test was originally written against sample catalog names (OPQ32r, General Ability Screen) but the app runs against real scraped names (SHL Occupational Personality Questionnaire (OPQ), Global Skills Assessment). Fixed by aligning test assertions with the production catalog.

Deployment

The project deploys via Render using the included `render.yaml` blueprint and `Dockerfile`. The free tier is sufficient: the evaluator allows up to 2 minutes for cold-start wake-up, and the service's local catalog retrieval keeps response times well under the 30-second per-call limit. Only one secret key is required: `DEEPSEEK_API_KEY`.

Final Tradeoff

The current system favors reliability, inspectability, schema correctness, and grounded local retrieval over highly semantic retrieval or richer but less controllable model-only behavior. This makes it a strong fit for the assignment's automated evaluator, where correctness and defensibility carry more weight than flashy orchestration.